

Understanding Mathematical Concepts via Go Language

Graph (Part 2)

Hyosang Kang (DGIST)

고 언어로 수학적 개념 이해하기

그래프 (파트 2)

강효상 (DGIST)

Testing Graphs

Implementations - Single Variable Function

```
type Single struct {  
    Interval  
    Eval func(float64) float64  
    Prec func(float64, float64) float64  
}
```

- `Single` is `Grapher`

```
func (s Single) Lift(p gocalc.Point) gocalc.Point {  
    x := p.Map(0).ToFloat()  
    return &SimpleVector{  
        x,  
        s.Eval(x),  
    }  
}  
  
func (s Single) Project(p gocalc.Point) gocalc.Point {  
    return p  
}
```

- Single is Continuous

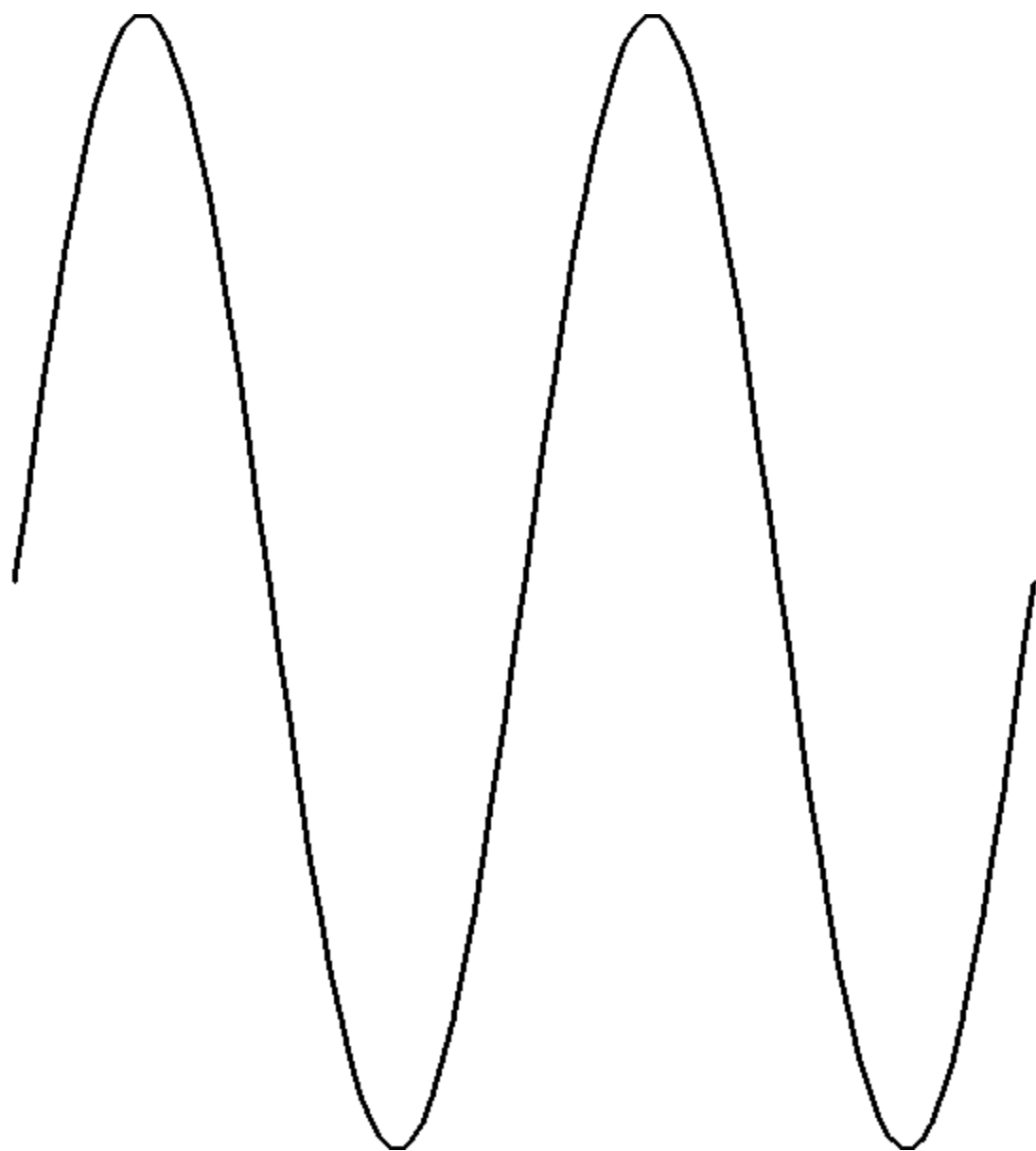
```
func (s Single) Eps(d gocalc.Real, x gocalc.Point) gocalc.Real {  
    return SimpleReal(s.Prec(d.ToFloat(), x.Map(0).ToFloat()))  
}  
  
func (s Single) Map(p gocalc.Point) gocalc.Vector {  
    if p.Len() != 1 {  
        panic("Single.Map: P dim mismatch")  
    }  
    x := p.Map(0).ToFloat()  
    return SimpleVector([]float64{s.Eval(x)})  
}
```

Testing Single - 1

$$f(x) = \sin(x)$$

```
func TestGraph1(t *testing.T) {  
    single := Single{  
        Interval: Interval{-2 * math.Pi, 2 * math.Pi},  
        Eval: func(x float64) float64 {  
            return math.Sin(x)  
        },  
        Prec: func(d, c float64) float64 {  
            return d  
        },  
    }  
}
```

```
opt := gocalc.GraphOption{
    Nsub: 100,
    Xmin: -7, Xmax: 7,
    Ymin: -1, Ymax: 1,
    Width: 600, Height: 600,
    Filename: "test_graph_1.png",
}
opt.Save(single)
}
```

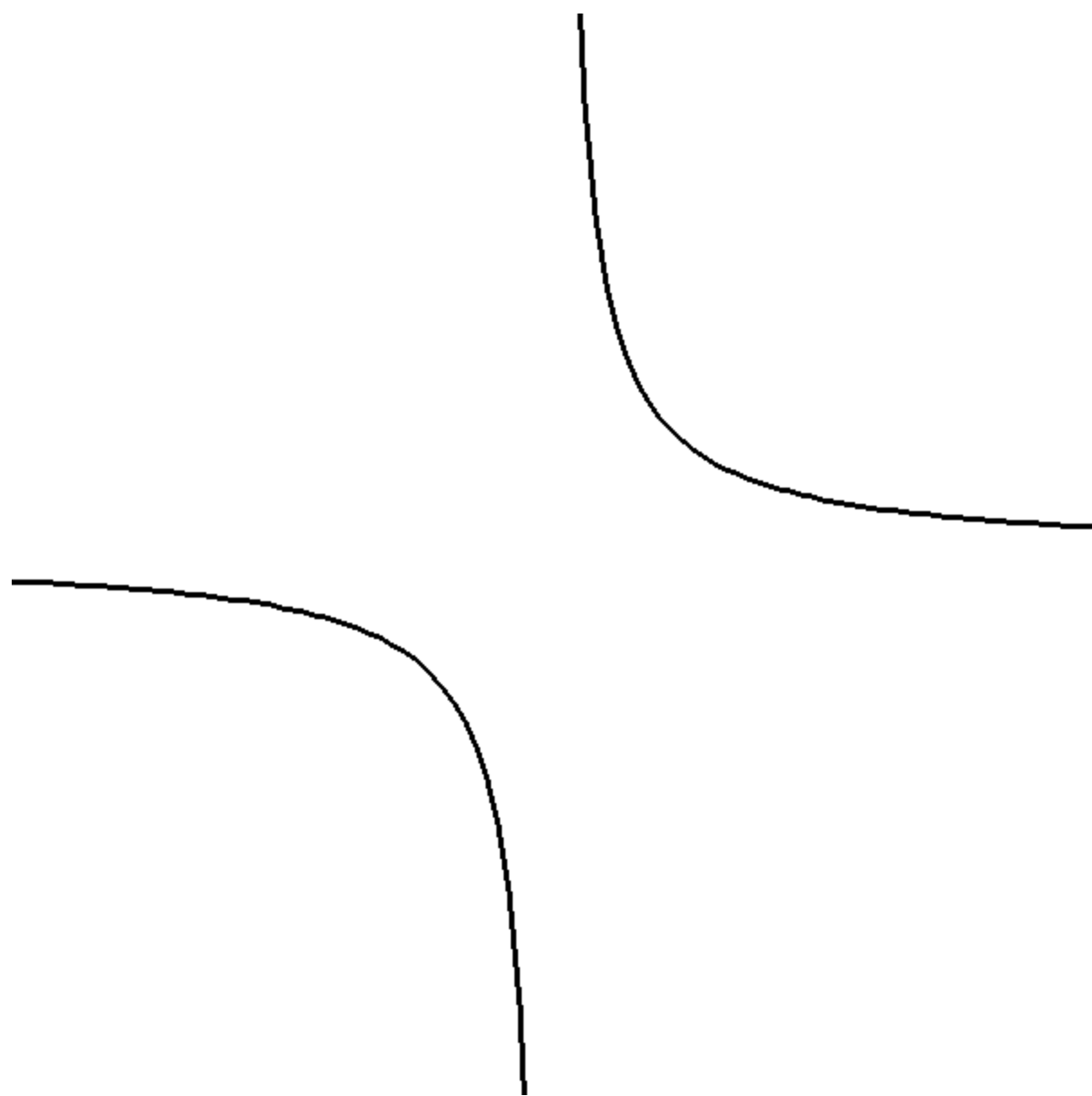



Testing Single - 2

$$f(x) = \frac{1}{x}$$

```
func TestGraph2(t *testing.T) {  
    single := Single{  
        Interval: Interval{-2, 2},  
        Eval: func(x float64) float64 {  
            if -1e-6 < x && x < 1e-6 {  
                return 0  
            }  
            return 1 / x  
        },  
        Prec: func(d, c float64) float64 {  
            return 1 / d  
        },  
    }  
}
```

```
opt := gocalc.GraphOption{
    Nsub: 100,
    Xmin: -2, Xmax: 2,
    Ymin: -10, Ymax: 10,
    Width: 600, Height: 600,
    Filename: "test_graph_2.png",
}
opt.Save(single)
}
```



Implementation - Two-Variable Function

```
type Double struct {  
    Rect  
    Eval func(float64, float64) float64  
}
```

- Double is Grapher

```
func (d Double) Lift(p gocalc.Point) gocalc.Point {  
    x, y := p.Map(0).ToFloat(), p.Map(1).ToFloat()  
    return SimpleVector{x, y, d.Eval(x, y)}  
}
```

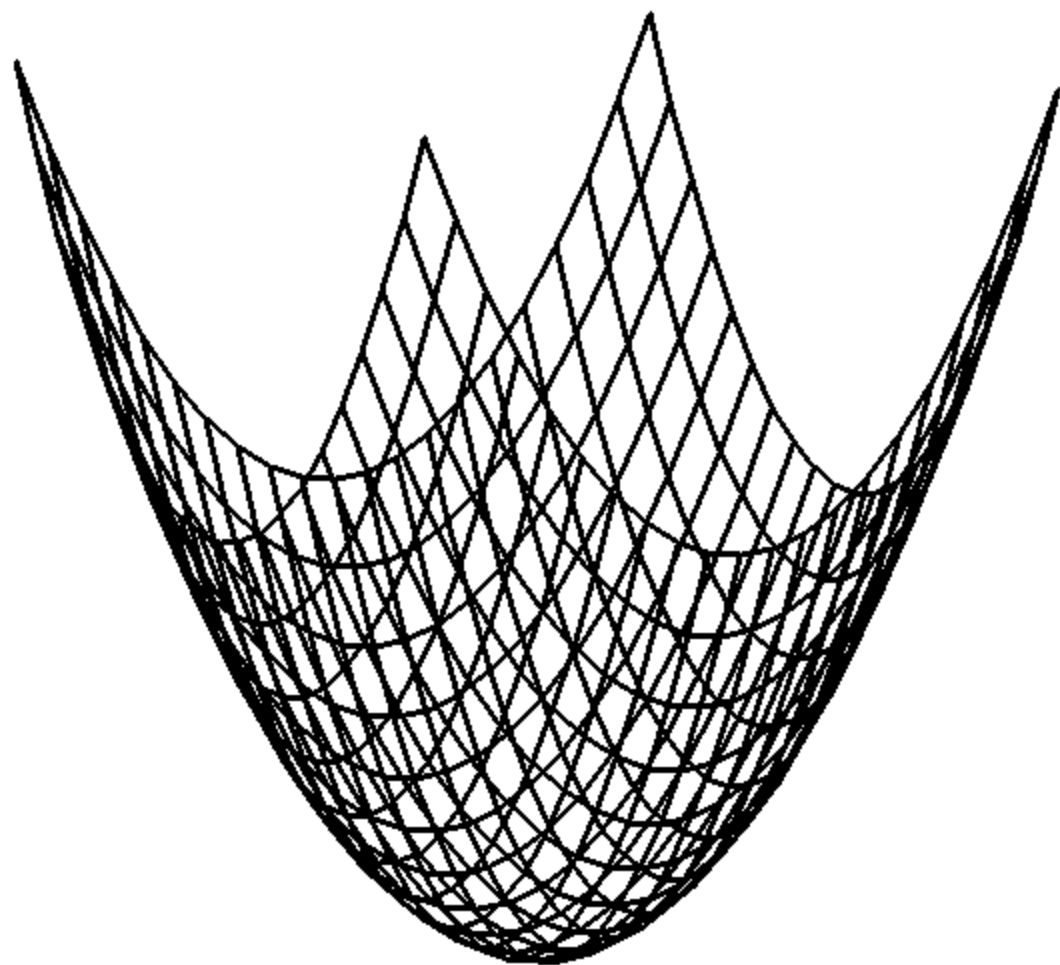
```
func (d Double) Project(p gocalc.Point) gocalc.Point {  
    t1, t2 := -1.0, -1.0  
    x, y, z := p.Map(0).ToFloat(), p.Map(1).ToFloat(), p.Map(2).ToFloat()  
    x, y = x*math.Cos(t1)-y*math.Sin(t1), x*math.Sin(t1)+y*math.Cos(t1)  
    y, _ = y*math.Cos(t2)-z*math.Sin(t2), y*math.Sin(t2)+z*math.Cos(t2)  
    return SimpleVector{x, y}  
}
```

Testing Double

$$f(x, y) = x^2 + y^2$$

```
func TestGraph3(t *testing.T) {  
    double := Double{  
        Rect: []Interval{  
            {-2 * math.Pi, 2 * math.Pi},  
            {-2 * math.Pi, 2 * math.Pi},  
        },  
        Eval: func(x, y float64) float64 {  
            return x*x + y*y  
        },  
    }  
}
```

```
opt := gocalc.GraphOption{
    Nsub: 20,
    Xmin: -10, Xmax: 10,
    Ymin: -10, Ymax: 80,
    Width: 600, Height: 600,
    Filename: "test_graph_3.png",
}
opt.Save(double)
}
```

Implementation - Surface

```
type Surface struct {  
    Rect  
    Local func(...float64) []float64  
}
```

- Surface is Grapher

```
func (s Surface) Lift(p gocalc.Point) gocalc.Point {
    x, y := p.Map(0).ToFloat(), p.Map(1).ToFloat()
    return SimpleVector(s.Local(x, y))
}

func (s Surface) Project(p gocalc.Point) gocalc.Point {
    dim := p.Len()
    x := make([]float64, dim)
    for i := range x {
        x[i] = p.Map(i).ToFloat()
    }
    theta := -.6
    for i := 0; i < dim-1; i++ {
        x[i], x[i+1] = x[i]*math.Cos(theta)-x[i+1]*math.Sin(theta), x[i]*math.Sin(theta)+x[i+1]*math.Cos(theta)
    }
    return SimpleVector{x[0], x[dim-1]}
}
```

- `Surface` will also implement `Manifold` and `Parametric` which will be explained later.

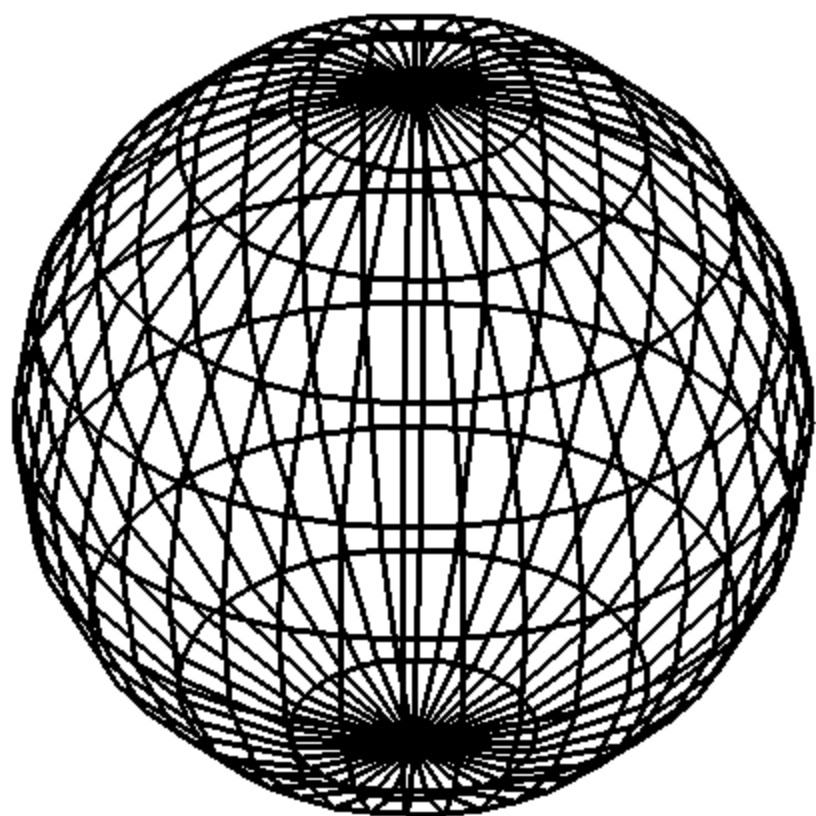
```
type Parametric interface {  
    Region  
    Map[Point, Vector]  
}  
  
type Manifold interface {  
    Locals() <-chan Parametric  
}
```

Testing Surface - 1

$$f(\theta, \phi) = (\sin(\theta) \cos(\phi), \sin(\theta) \sin(\phi), \cos(\theta))$$

```
func TestGraph5(t *testing.T) {  
    s := Surface{  
        Rect: []Interval{  
            {0, math.Pi},  
            {0, 2 * math.Pi},  
        },  
        Local: func(x, y float64) []float64 {  
            return []float64{  
                math.Sin(y) * math.Cos(x),  
                math.Sin(y) * math.Sin(x),  
                math.Cos(y),  
            }  
        },  
    }  
}
```

```
opt := gocalc.GraphOption{  
    Nsub: 20,  
    Xmin: -1.5, Xmax: 1.5,  
    Ymin: -1.5, Ymax: 1.5,  
    Width: 600, Height: 600,  
    Filename: "test_graph_5.png",  
}  
opt.Save(s)  
}
```



Testing Surface - 2

- Klein bottle

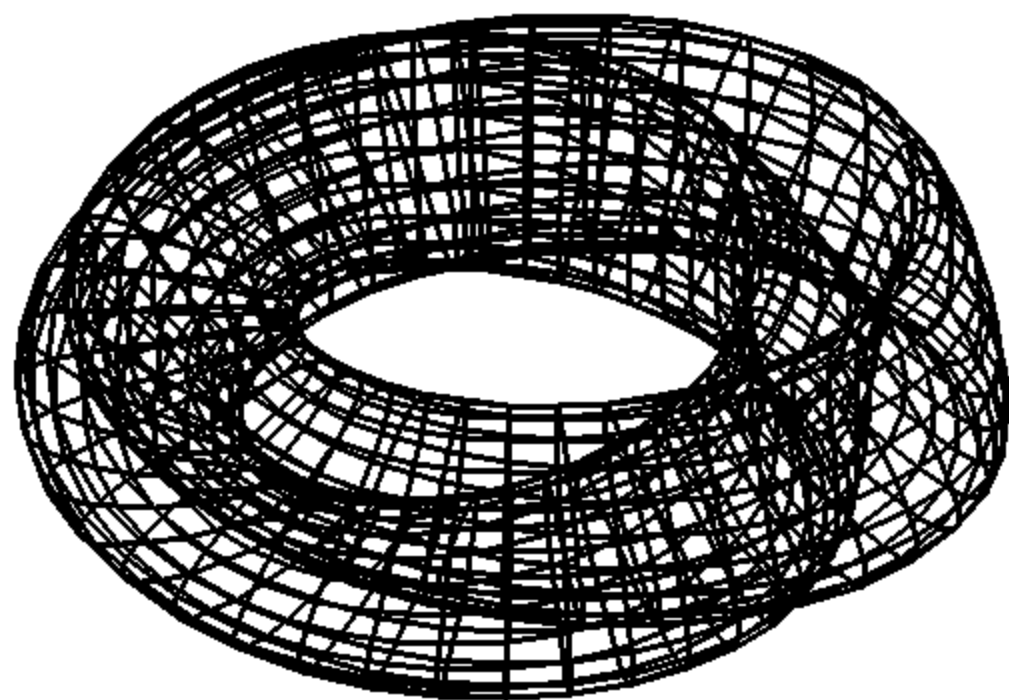
$$f(\theta, \phi) = \begin{pmatrix} (3 + \cos(\theta/2) \sin(\phi) - \sin(\theta/2) \sin(2\phi)) \cos(\theta), \\ (3 + \cos(\theta/2) \sin(\phi) - \sin(\theta/2) \sin(2\phi)) \sin(\theta), \\ \sin(\theta/2) \sin(\phi) + \cos(\theta/2) \sin(2\phi) \end{pmatrix}$$


```

func TestGraph4(t *testing.T) {
    s := Surface{
        Rect: []Interval{
            {0, 2 * math.Pi},
            {0, 2 * math.Pi},
        },
        Local: func(x, y float64) []float64 {
            C, S := math.Cos, math.Sin
            return []float64{
                (3 + C(x/2)*S(y) - S(x/2)*S(2*y)) * C(x),
                (3 + C(x/2)*S(y) - S(x/2)*S(2*y)) * C(x),
                S(x/2)*S(y) + C(x/2)*S(2*y),
            }
        },
    }
}

```

```
opt := gocalc.GraphOption{
    Nsub: 40,
    Xmin: -5, Xmax: 5,
    Ymin: -5, Ymax: 5,
    Width: 600, Height: 600,
    Filename: "test_graph_4.png",
}
opt.Save(s)
}
```



This video was produced with the support of CTL in DGIST.

Production: Hyosang Kang

Editing: Jongwoo Baek

본 동영상은 DGIST의 CTL의 지원을 받아 제작되었습니다.

제작: 강효상

편집: 백종우